THE INTERFACE QUESTION

When Codex and Claude Code Need to Talk

Inside ACC, a local channel for independent coding sessions

Mykola Maksymenko
Independent creator of ACC

THE WORKFLOW

Independently opened sessions

Codex	API contract, own conversation, own permissions
Claude Code	Form example, own conversation, own permissions
ACC	Shared coordination facts on this machine

RUNNING DEMO

One interface question

The form needs the API’s duplicate-email contract

```
POST /register
request: { email, password }
409:     { code: "EMAIL_TAKEN" }
```

Question, answer, resulting artifact, exact receipt history

DURABLE FIRST

Record before delivery

1. Commit message + queued receipts
2. Select one eligible recipient generation
3. Ask the client adapter to offer it
4. Record offered after acceptance

A failed offer leaves the durable inbox intact

OBSERVABLE FACTS

What a receipt proves

`queued`	Message and receipt committed
`offered`	Transport accepted bytes, or hook carried body
`retrieved`	Recipient fetched the body
`acknowledged`	Recipient acknowledged or replied

Task completion needs its own evidence

Manual inbox reads can skip offered

REPRODUCIBLE REPLAY

Retry without a second question

```
same retry key + equivalent content
```

```
original message id
one inbox item
```

Different content with the same key is an error

An accepted transport offer can still be displayed again after a crash

Equivalent send retries and repeat display are different boundaries

OWNERSHIP

Participant and session generation

Participant The stable message address

Session One current opening of that participant

Generation The owner token for that opening

Public visibility does not establish ownership

SHARED FILE SCOPE

Claims have a limited job

```
Codex:  file:api/**
Claude: file:ui/**

overlap: conflict, exit 5
```

A leased reservation, advisory by default

Guards cover only certified client mutation paths

CLIENT BOUNDARY

Support, reachability and consent

Captured support	What an installed client demonstrably did
Current reachability	One generation, handshake and valid lease
Recipient consent	Live delivery policy, off by default
Fallback	Durable inbox and supported normal-turn delivery

Native acceleration is experimental and can spend model tokens

TRUST BOUNDARY

Peer text remains untrusted

A delivered request carries context.
The recipient keeps its own authority.

Attributed envelope, escaped rendering, bounded context

Models can still mishandle persuasive text

HOOK BUDGET

Coordination can fail open

A slow coordination hook must let the coding session continue

Report degradation and preserve the durable records

Refusing an unsafe store mutation is a separate boundary

Continuing without fresh context can miss a dependency

A REAL FAILURE

A hook reread the backlog

Copied workspace	Before	After
ACC	0.96–1.05 s	0.29–0.33 s
newsmill-ai	1.10–1.30 s	0.30–0.34 s
ClaudeProbe	3.21–3.80 s	0.39–0.41 s

Repeated backlog reads consumed the hook budget

Historical turn measurements, macOS / Node 26.5.1
3 alternating rounds, copied real data home, load average 10–27
Retained development evidence, not a new benchmark

SOURCE WALKTHROUGH

One shared read per turn

```
const snapshot = await
context.service.store.snapshot(context.descriptor.id);
const sync = await context.service.sync({ sessionId:
binding.accSessionId,
  cursor: null, scope: "delta", snapshot });
```

Closing status only on request

Heartbeat loads its own session record

A turn still reads the backlog once

VERIFICATION

The installed boundary is the test

Package, client, message, observable result

Installed entrypoints and client activation
Versioned captures and declared limits
Mutations that trigger the intended failure

A model stub checks integration behavior, not reasoning quality

A successful config write is weaker evidence than a working client

CURRENT SCOPE

Current boundaries

- One machine and operating-system user
- Explicit messages, no raw transcript collection
- Advisory claims unless a relevant guard is available
- Client evidence, handshake and consent for live delivery
- Model decisions and task results need separate evaluation

THE EXPERIMENT

One real dependency

Two clients.
One feature.
A question they need to exchange.

Where did ACC help?
Where did you still carry a message yourself?

`github.com/automatis-tools/agents-can-communicate`